

IDS Pipeline Metrics Monitoring Guide

Overview

The IDS pipeline includes a comprehensive metrics monitoring system that tracks performance in real-time. Metrics are automatically logged during pipeline execution and can be viewed via a live dashboard or exported for analysis.

Key Features:

- Real-time latency tracking (P50/P95/P99 percentiles)
- Throughput monitoring (events/sec, bytes/sec)
- ML model performance metrics
- System resource usage
- Error tracking
- Thread-safe, low-overhead design (< 1% CPU)

Architecture Overview

Data Flow

Network Traffic



STAGE 1: PACKET CAPTURE (DPDK or AF_PACKET)



[Suricata NIDS]

Generates EVE JSON logs
/var/log/suricata/eve.json



STAGE 2: KAFKA BRIDGE

- Monitors EVE JSON logs
- Parses events (flow, alert, http, dns, etc.)
- Publishes to Kafka topic: "suricata-events"

METRICS LOGGED:

- File read latency
- JSON parse latency
- Kafka publish latency
- Throughput (events/sec)



[Kafka Topic: suricata-events]

↓

STAGE 3: ML CONSUMER

1. Consume from Kafka
2. Extract 65 CICIDS2017 features
3. ML inference (Random Forest/LightGBM)
4. Combine with Suricata alerts
5. Publish enhanced alerts

📊 METRICS LOGGED:

- Kafka consume latency
- Feature extraction latency
- ML inference latency
- End-to-end latency
- Throughput (predictions/sec)
- ML confidence scores
- Prediction distribution

↓

[Kafka Topic: ids-alerts]

How Metrics Are Calculated

1. Latency Measurement

Latency is measured using a timer that records the duration of operations:

Process:

Operation Start

↓

[Record start_time = current_timestamp]

↓

[Your code executes]

↓

[Record end_time = current_timestamp]

↓

[Calculate: latency = end_time - start_time]

↓

[Store in thread-safe buffer]

Statistical Aggregation:

- Metrics are buffered in memory (up to 1000 samples)
- When statistics are requested, samples are sorted
- Percentiles calculated by position in sorted array:

- **P50 (median):** Value at 50th percentile → 50% of operations faster
- **P95:** Value at 95th percentile → 95% of operations faster
- **P99:** Value at 99th percentile → 99% of operations faster

Why percentiles matter:

- Mean (average) can be skewed by outliers
- P95/P99 show "typical worst case" performance
- P99 > 100ms indicates outliers that need investigation

2. Throughput Calculation

Throughput measures events/bytes processed over time windows:

Formula:

```
events_per_second = event_count / time_window_seconds
bytes_per_second = byte_count / time_window_seconds
```

Implementation:

- Sliding windows track last N seconds of activity
- Rates calculated continuously
- Both instantaneous and cumulative statistics maintained

3. ML Metrics

Tracks machine learning model performance:

- **Inference time:** Duration of model.predict() call
- **Confidence scores:** Model's certainty in predictions
- **Prediction distribution:** Breakdown of attack types detected
 - BENIGN (normal traffic)
 - DoS, DDoS, PortScan, etc.

PROF

4. Thread-Safe Buffering

All metrics use thread-safe queues with locks:

```
[Metric Event]
↓
[Acquire lock]
↓
[Append to buffer (deque with maxlen=1000)]
↓
[Update running statistics]
↓
[Release lock]
```

↓
[Background thread flushes to disk every 30s]

Output files:

- `logs/metrics/metrics_YYYYMMDD.jsonl` (JSON Lines, one metric per line)
- `logs/metrics/metrics_YYYYMMDD.csv` (CSV format for spreadsheet analysis)

Integration with DPDK and AF_PACKET Modes

The metrics system is **mode-agnostic** and works identically with both capture methods:

DPDK Mode Integration

```
graph TD
    A[DPDK PMD (Poll Mode Driver)] --> B[Suricata with DPDK Workers]
    B -- "[High-speed packet processing]" --> C[EVE JSON Output]
    C --> D["Metrics Logger (Same Pipeline)"]
    D --> E["- Bridge: File → Kafka metrics"]
    D --> F["- ML Consumer: Processing metrics"]
```

DPDK-Specific Considerations:

- Higher throughput requires more frequent metrics sampling
- Lower latency values expected (< 5ms P95 for packet capture)
- More network interface statistics available

AF_PACKET Mode Integration

```

Linux Kernel AF_PACKET
    ↓
Suricata with AF_PACKET Workers
    ↓ [Standard packet processing]
    ↓
EVE JSON Output
    ↓


Metrics Logger (Same Pipeline)
    - Bridge: File → Kafka metrics
    - ML Consumer: Processing metrics


```

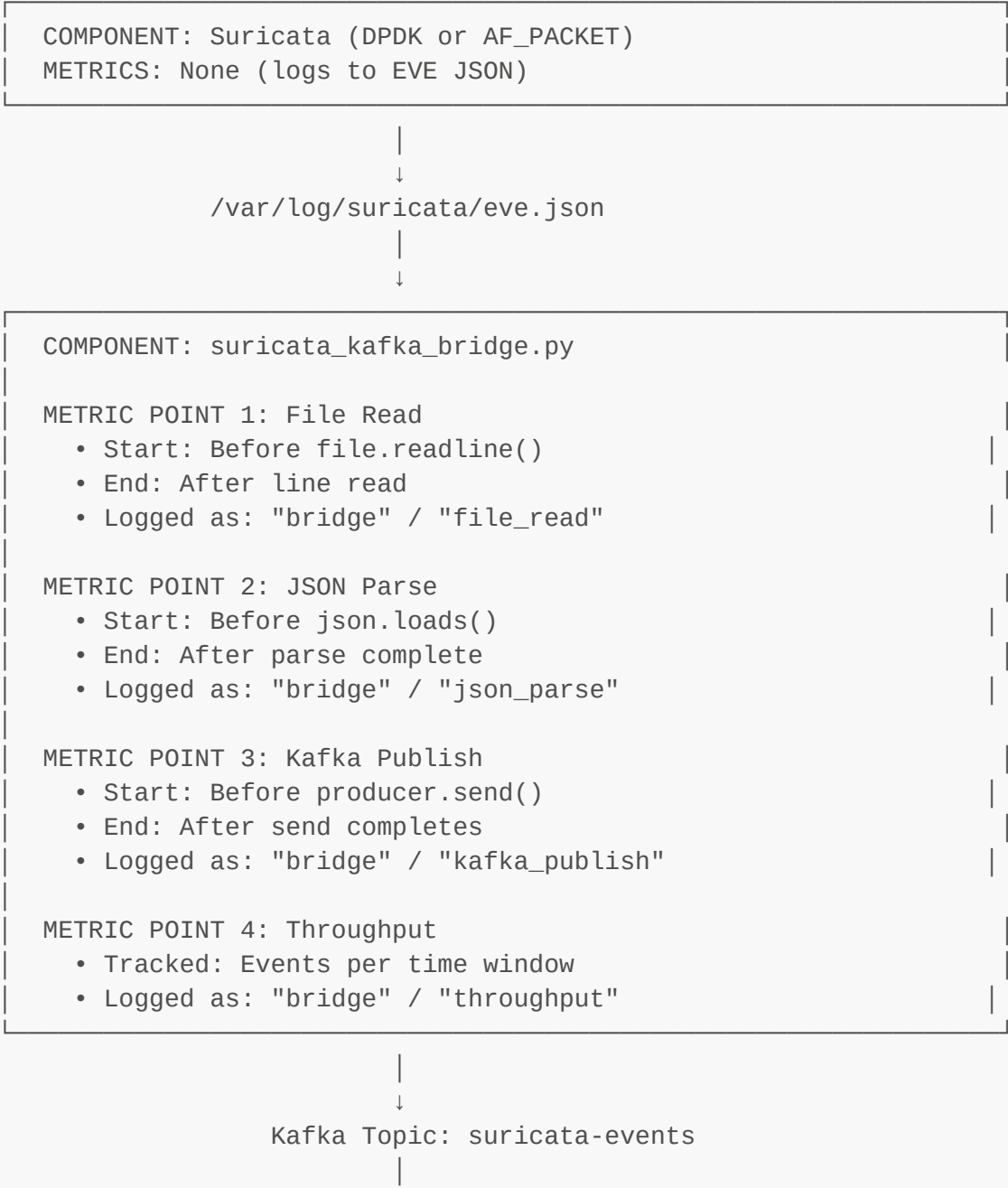
AF_PACKET-Specific Considerations:

- Moderate throughput suitable for most deployments
- Slightly higher latency (5-20ms P95 for packet capture)
- CPU scheduling affects latency more than DPDK

Key Point: The metrics collection is **downstream** of the packet capture mechanism. Whether using DPDK or AF_PACKET, the same metrics are collected starting from the EVE JSON stage.

Metrics Collection Points

Detailed Flow Diagram





COMPONENT: ml_kafka_consumer.py

METRIC POINT 5: Kafka Consume

- Start: Before consumer.poll()
- End: After message received
- Logged as: "ml_consumer" / "kafka_consume"

METRIC POINT 6: Feature Extraction

- Start: Before extract_features()
- End: After 65 features computed
- Logged as: "ml_consumer" / "feature_extraction"

METRIC POINT 7: ML Inference

- Start: Before model.predict()
- End: After prediction returned
- Logged as: "ml_consumer" / "ml_inference"
- Additional: confidence, prediction class

METRIC POINT 8: End-to-End

- Start: When message first received
- End: After alert published to Kafka
- Logged as: "ml_consumer" / "end_to_end"

METRIC POINT 9: Throughput

- Tracked: Predictions per time window
- Logged as: "ml_consumer" / "throughput"



Kafka Topic: ids-alerts

Usage Instructions

Quick Start

Option 1: Automatic Startup (Recommended)

```
cd /home/sujay/Programming/IDS
sudo ./start_ids_with_metrics.sh
```

This starts all components in tmux windows. Use **Ctrl+B** then **0-3** to switch between:

- Window 0: Suricata
- Window 1: Kafka Bridge
- Window 2: ML Consumer
- Window 3: Metrics Dashboard

Option 2: Monitor Already Running Pipeline ☆ Most Common

If your pipeline is already running (e.g., from `run_afpacket_mode.sh`):

```
# Terminal 1: Pipeline already running
sudo ./run_afpacket_mode.sh # Already started

# Terminal 2: Monitor metrics
./monitor_metrics.sh # Launch dashboard
```

 **Tip:** See [MONITORING_SETUP.md](#) for detailed monitoring instructions.

Option 3: Manual Startup

```
# Terminal 1: Start Suricata (choose mode)
sudo ./run_dpdk_mode.sh # For DPDK mode
# OR
sudo ./run_afpacket_mode.sh # For AF_PACKET mode

# Terminal 2: Start Kafka Bridge
cd dpdk_suricata_ml_pipeline
python3 src/suricata_kafka_bridge.py

# Terminal 3: Start ML Consumer
python3 src/ml_kafka_consumer.py

# Terminal 4: View Metrics Dashboard
./monitor_metrics.sh # Simplified monitoring
# OR
./dpdk_suricata_ml_pipeline/scripts/metrics_dashboard.py # Direct script
```

PROF

Dashboard Output

The dashboard displays real-time metrics:

IDS Pipeline Metrics Dashboard

LATENCY METRICS

Component: ml_consumer | Operation: ml_inference

Count: 1,234 samples

Mean: 12.3 ms

P50: 11.2 ms ← Median (typical request)

P95: 18.9 ms ← 95% complete under this time

P99: 25.7 ms ← 99% complete under this time



THROUGHPUT METRICS

Component: ml_consumer

Events/sec: 82.5

Total Events: 1,234



ML METRICS

Model: random_forest_2017

Avg Inference: 12.3 ms

Avg Confidence: 0.89

Prediction Distribution:

BENIGN: 1,100 (89.1%)

DoS: 89 (7.2%)

PortScan: 45 (3.6%)



SYSTEM METRICS

CPU Usage: 45.2%

Memory: 1,234 MB

Network RX: 10.5 MB

Network TX: 2.3 MB

[Refreshes every 5 seconds]

Viewing Metrics Files

Metrics are written to `logs/metrics/` directory:

View real-time metrics:

```
tail -f logs/metrics/metrics_$(date +%Y%m%d).jsonl | jq '.'
```

View specific metric types:

```
# Latency metrics only
cat logs/metrics/metrics_*.jsonl | jq 'select(.metric_type ==
"latency")'

# ML inference metrics
cat logs/metrics/metrics_*.jsonl | jq 'select(.metric_type ==
"ml_inference")'
```


Calculate statistics:

```
# Average ML inference latency
cat logs/metrics/metrics_*.jsonl | \
jq -s 'map(select(.metric_type == "latency" and .operation ==
"ml_inference")) |
      map(.latency_ms) | add / length'

# Find slowest operations
cat logs/metrics/metrics_*.jsonl | \
jq 'select(.metric_type == "latency")' | \
jq -s 'sort_by(.latency_ms) | reverse | .[0:10]'
```

Export to spreadsheet:

```
# CSV file is automatically generated
libreoffice logs/metrics/metrics_$(date +%Y%m%d).csv
```

Performance Benchmarks

Expected Values

Metric	Good	Acceptable	Poor
Kafka consume (P95)	< 5ms	< 20ms	> 50ms
Feature extraction (P95)	< 10ms	< 30ms	> 100ms
ML inference (P95)	< 15ms	< 50ms	> 200ms
End-to-end (P95)	< 50ms	< 100ms	> 500ms
Throughput	> 100 evt/s	> 50 evt/s	< 10 evt/s
CPU usage	< 60%	< 80%	> 90%

Mode-Specific Performance

DPDK Mode:

- Higher throughput: 200-1000+ events/sec
- Lower latency: P95 typically 30-50ms end-to-end
- CPU usage: 40-70% (dedicated cores)

AF_PACKET Mode:

- Moderate throughput: 50-200 events/sec
- Moderate latency: P95 typically 50-100ms end-to-end

- CPU usage: 50-80% (shared cores)

Troubleshooting

Dashboard shows no data

Check:

```
ls -lh logs/metrics/
```

Should see: `metrics_YYYYMMDD.jsonl` file

If missing: Metrics logger not initialized in pipeline code

High latency (P99 > 1000ms)

Identify bottleneck:

```
cat logs/metrics/metrics_*.jsonl | \
  jq 'select(.metric_type == "latency")' | \
  jq -s 'group_by(.operation) |
        map({op: .[0].operation, avg: (map(.latency_ms) | add /
length))} |
        sort_by(.avg) | reverse'
```

Common causes:

- **Feature extraction slow:** Optimize feature calculations
- **ML inference slow:** Use lighter model or batch predictions
- **Kafka slow:** Increase partitions, check network latency
- **CPU saturation:** Scale horizontally or reduce workload

Low throughput (< 10 events/sec)

Check end-to-end latency:

- High latency + low throughput = bottleneck exists
- Low latency + low throughput = insufficient traffic

Solutions:

- Profile slow operations
- Increase Kafka consumer parallelism
- Use batch processing for ML inference
- Optimize feature extraction code

Metrics files growing too large

Automatic rotation:

- Files rotate daily: `metrics_20251026.jsonl` → `metrics_20251027.jsonl`

Manual cleanup:

```
# Compress old files
cd logs/metrics
gzip metrics_$(date -d '1 day ago' +%Y%m%d).jsonl

# Delete files older than 7 days
find logs/metrics -name "metrics_*.jsonl" -mtime +7 -delete
```

Key Concepts Summary

Metrics Types:

- Latency:** Time operations take (milliseconds)
- Throughput:** Events processed per second
- ML Metrics:** Model performance and predictions
- System Metrics:** CPU, memory, network usage

Percentiles Explained:

- P50 (median):** Half faster, half slower
- P95:** 95% faster than this ← **Most important for performance**
- P99:** 99% faster than this ← **Catches outliers**

Integration Points:

- Metrics collected **after** Suricata processing
- Works identically with DPDK or AF_PACKET
- Thread-safe, low-overhead design
- Automatic buffering and flushing

Files:

- JSON Lines: `logs/metrics/metrics_YYYYMMDD.jsonl`
- CSV: `logs/metrics/metrics_YYYYMMDD.csv`
- Dashboard script: `dppdk_suricata_ml_pipeline/scripts/metrics_dashboard.py`
- Startup script: `start_ids_with_metrics.sh`

Common Commands

```
# Start pipeline with metrics
sudo ./start_ids_with_metrics.sh

# Check status
ps aux | grep -E 'suricata|bridge|consumer|dashboard'

# Stop all components
sudo pkill -f suricata
pkill -f suricata_kafka_bridge
pkill -f ml_kafka_consumer
pkill -f metrics_dashboard

# View logs
tail -f logs/ml/ml_consumer.log
tail -f logs/metrics/metrics_*.jsonl | jq '.'

# Generate test traffic
python3 tests/test_benign_traffic.py
python3 tests/test_attack_generator.py
```

For detailed API documentation and code integration examples, refer to [METRICS_README.md](#) and [METRICS_INTEGRATION_GUIDE.md](#).